RIPASSO C
Parte 3

(Slides cortesemente concesse in uso dal Prof. Fraschini)

Puntatori e strutture

```c
#include <stdio.h>
#define DIM_NOME 50

int main()
{
  struct studente
  {
      char nome[DIM_NOME];
      char cognome[DIM_NOME];
      int matricola;
  };

  struct studente stud;
  struct studente *punt_stud;

  punt_stud=&stud;

  printf("Inserisci nome studente: \n");
  scanf("%s",stud.nome);
  printf("Inserisci cognome studente: \n");
  scanf("%s",stud.cognome);
  printf("Inserisci matricola studente: \n");
  scanf("%d",&stud.matricola);

  printf("Dati dello studente inserito:\n");
  printf("%s %s %d\n",(*punt_stud).nome,(*punt_stud).cognome,(*punt_stud).matricola);

  return 0;
}
```

Accedo ai campi così…

(*punt_stud).nome

Puntatori e strutture

```c
#include <stdio.h>
#define DIM_NOME 50

int main()
{
  struct studente
  {
      char nome[DIM_NOME];
      char cognome[DIM_NOME];
      int matricola;
  };

  struct studente stud;
  struct studente *punt_stud;

  punt_stud=&stud;

  printf("Inserisci nome studente: \n");
  scanf("%s",stud.nome);
  printf("Inserisci cognome studente: \n");
  scanf("%s",stud.cognome);
  printf("Inserisci matricola studente: \n");
  scanf("%d",&stud.matricola);

  printf("Dati dello studente inserito:\n");
  printf("%s %s %d\n",punt_stud->nome,punt_stud->cognome,punt_stud->matricola);

  return 0;
}
```

o così…

punt_stud->nome

Puntatori e strutture

```c
#include <stdio.h>
#define DIM_NOME 50

int main()
{
  struct studente
  {
      char nome[DIM_NOME];
      char cognome[DIM_NOME];
      int matricola;
  };

  struct studente stud;
  struct studente *punt_stud;

  punt_stud=&stud;

  printf("Inserisci nome studente: \n");
  scanf("%s",punt_stud->nome);
  printf("Inserisci cognome studente: \n");
  scanf("%s",punt_stud->cognome);
  printf("Inserisci matricola studente: \n");
  scanf("%d",&punt_stud->matricola);

  printf("Dati dello studente inserito:\n");
  printf("%s %s %d\n",stud.nome,stud.cognome,stud.matricola);

  return 0;
}
```

o così.

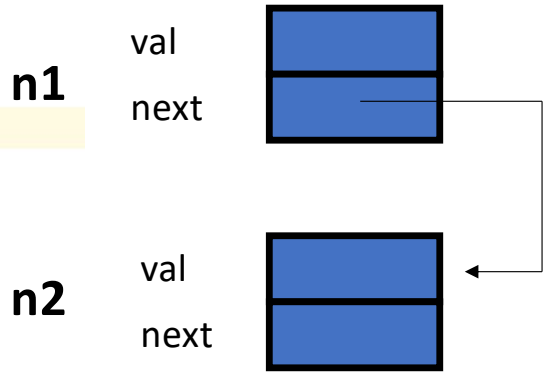
stud.nome

Liste concatenate

- *Puntatori a strutture* e *strutture con puntatori* sono di fondamentale importanza
- Permettono di creare **liste concatenate** e **alberi**

Liste concatenate

```c
#include <stdio.h>

int main()
{
    struct valori
    {
        int val;
        struct valori *next;

    };

    struct valori n1, n2;

    n1.next=&n2; //il puntatore nella struttura n1 punta alla struttura n2

    return 0;
}
```



Liste concatenate

```c
#include <stdio.h>

int main()
{
    struct valori
    {
        int val;
        struct valori *next;

    };

    struct valori n1, n2, n3;
    int x;

    n1.val=10;
    n2.val=20;
    n3.val=30;

    n1.next=&n2;
    n2.next=&n3;

    x=n1.next->val;
    printf("x = %d\n",x);
    printf("%d\n",n2.next->val);

    return 0;
}
```

```
x = 20
30
```

Liste concatente

- Permettono di eliminare facilmente elementi da un elenco di valori
- Se volessimo **eliminare** n2 dalla lista:

n1.next = n2.next;

- Per **inserire** un nuovo elemento n2_bis (tra n2 e n3):

n2_bis.next=n2.next;

n2.next=&n2_bis;

Gli elementi di una lista non devono essere memorizzati in sequenza nella memoria!

Liste concatente

- Importanza di un puntatore esterno che punti al **primo elemento** di una lista

struct valori *punt_lista=&n1; //con n1 già definito!

- Importanza identificatore **fine lista**
- **Puntatore nullo (puntatore NULL) ->** puntatore che non punta a nulla

n3.next = NULL;

Scorri lista

```c
#include <stdio.h>

int main()
{
  struct valori
  {
    int val;
    struct valori *next;
  };
  struct valori n1,n2,n3;
  struct valori *punt_lista=&n1;

  n1.val=10;
  n1.next=&n2;

  n2.val=20;
  n2.next=&n3;

  n3.val=30;
  n3.next=NULL;

  while(punt_lista != NULL)
  {
    printf("%d ",punt_lista->val);
    punt_lista=punt_lista->next;
  }

  return 0;
}
```

```
10 20 30
```